Choosing Retrieval-Augmented Generation Strategies by Domain: An Empirical Comparison

Deeparsh Singh Dang
Thompson Rivers University
deeparshsingh10@gmail.com

Gursahib Singh
Thompson Rivers University
gursahib99888@gmail.com

Jaspreet Kaur
Assistant Professor
Thompson Rivers University

Ghazanfar Latif
Assistant Professor
Thompson Rivers University

***Abstract*—Retrieval-Augmented Generation (RAG) now has many variants, but a practitioner who must choose one for a specific domain rarely has a comparison run under matched conditions. This paper reports an empirical study of seven strategies for answering questions over a document collection: four retrieval pipelines (naive dense retrieval, hybrid lexical and dense retrieval, hypothetical-document embeddings, and cross-encoder reranking) and three strategies without retrieval (full-context injection, recursive summarization, and sliding-window reasoning). All seven were evaluated on the same 8,000-token corpus and 34 hand-authored questions of five types (sanity checks, single-fact recall, multi-hop reasoning, distractor robustness, and unanswerable questions), with three seeds, bootstrap confidence intervals, and corrected significance tests. The experiment was run on a proprietary generator, Gemini 3 Flash, and replicated on an open-weights generator, GLM 5.3 Flash. The four retrieval pipelines gave nearly the same results on both generators (F_1 differences of at most 3.8 points), whereas the choice of strategy on a single generator changed F_1 by up to 67.5 points. Full-context injection gave the highest F_1 on both generators (0.881 and 0.915) when the corpus fit in the prompt, provided the reasoning model had enough token budget to reason; without that budget its F_1 fell to 0.490. Question-agnostic summarization lost specific facts on both generators, and a strict refusal instruction gave perfect refusal accuracy for every retrieval pipeline. From these results by question type we derive recommendations for medical, legal, educational, financial, software, and internal knowledge-base applications. The evaluation software is released as open source so that the study can be repeated on other corpora and models.**

***Index Terms*—Retrieval-Augmented Generation, empirical evaluation, question answering, domain-specific recommendations, cross-model replication, long-context language models, reasoning language models.**

I. Introduction

Retrieval-Augmented Generation (RAG) [1], [2] is the standard way to ground Large Language Models (LLMs) in private or domain-specific documents. Within five years the technique has grown from a single method into a family of variants, including HyDE [3], GraphRAG [4], RAPTOR [5], Self-RAG [6], hybrid retrieval [7], and cross-encoder reranking [8], [9], each addressing a specific weakness of earlier designs. At the same time, the assumption that motivated RAG, that the documents will not fit in the model's context window, is weakening as context windows grow [10]. Whether retrieval is still needed when the documents fit in the prompt is therefore an empirical question.

Practitioners who must choose a strategy for a specific domain get little help from this literature. A medical question-answering system has different requirements from a legal research tool or an educational tutor: recall of single facts is critical when answers depend on drug names or statute numbers, multi-hop reasoning matters when conclusions combine several documents, and refusing to answer is essential whenever a wrong answer is worse than no answer. Published RAG variants are usually reported in isolation, on benchmarks chosen to show their strengths, and often against different baselines. Comparisons under matched conditions (the same generator, chunking, prompts, and grading) that separate results by type of question are rare. A second open issue is how much such a comparison depends on the generator: if the ranking of strategies changed with every model, a comparison would have to be repeated for each one.

This paper reports a controlled empirical study designed to answer three research questions:

- **RQ1.** How do seven RAG and non-retrieval strategies compare in answer quality, refusal accuracy, cost, and latency, and on which types of question does each succeed or fail?
- **RQ2.** Do these results hold when the generator changes from a proprietary model to an open-weights model?
- **RQ3.** Which strategy should be used for which domain or use case?

A. Contributions

The paper makes the following contributions.

1) **A controlled comparison of seven strategies** (Naive, Hybrid, HyDE, and Cross-Encoder Reranker RAG; Full-Context Injection, Recursive Summarization, and Sliding-Window Reasoning) on one corpus and question set, broken down by five question types, with three seeds, bootstrap confidence intervals, and Holm-corrected significance tests (RQ1).
2) **A cross-model replication** on an open-weights generator, which shows that the retrieval pipelines behave almost identically on both generators while multi-step and long-prompt strategies depend on the model, and which identifies the reasoning-token budget as a setting that can decide the outcome (RQ2).

The code, evaluation data, and run artifacts used in this study are available at https://github.com/Singh-Gursahib/RAG-Approaches.

3) **Use-case recommendations** for six application areas, derived from the results by question type, with the evidence behind each recommendation stated and marked as measured or inferred (RQ3).
4) **The evaluation software used to run the study**, released as open source together with the corpus, questions, and every run artifact, so that the study can be repeated on other corpora and generators.

The remainder of the paper is organized as follows. Section II reviews related work. Section III describes the research design and the stages of the study. Section IV explains how the seven strategies and the evaluation harness were implemented. Section V specifies the corpus, questions, metrics, and generators. Section VI reports the results on both generators. Section VII states the research findings, answers the research questions, and gives the recommendations. Section VIII discusses limitations and future work, and Section IX concludes.

II. Related Work

A. Retrieval-Augmented Generation

RAG [1] couples a parametric language model with a non-parametric document store that is searched at inference time with a dense passage retriever [2]. Dense retrievers are typically trained as sentence encoders; Sentence-BERT [11] introduced the siamese fine-tuning method used by the `sentence-transformers` library and by the embedding model in this study, `all-MiniLM-L6-v2`.

Two directions of improvement are widely studied. First, retrieval quality can be improved by combining lexical [12] and dense signals with Reciprocal Rank Fusion [7], or by adding a cross-encoder reranker [8], [9] that scores each query and passage together; late-interaction models such as ColBERT [13] offer a less expensive alternative. Second, query reformulation methods such as HyDE [3] reduce the difference in wording between a question and its answer by first generating a hypothetical answer and retrieving with its embedding.

A separate line of work builds structured indexes. GraphRAG [4] extracts a graph of entities and relations and precomputes summaries of its communities; RAPTOR [5] builds a recursive tree of summaries. Self-RAG [6] trains the model to decide when to retrieve. These methods are outside the scope of this study (Section VIII).

B. Long-Context Models

Current flash-tier models [10] accept on the order of 10^6 input tokens, which raises the question of whether retrieval is still needed once the corpus fits in the prompt. Liu et al. [14] report that accuracy depends on where the evidence appears in a long prompt (the "lost in the middle" effect), so placing a whole corpus in the prompt has failure modes of its own. Many current models also reason before answering, which adds a second budget, the tokens spent reasoning, that interacts with long prompts (Section VI-D).

C. Domain-Specific RAG Evaluation

MedRAG [15] evaluates RAG configurations on the MIRAGE benchmark of 7,663 medical questions from five datasets and shows how the choice of retriever interacts with model scale. LegalBench [16] provides 162 legal-reasoning tasks written by legal professionals, and LegalBench-RAG [17] evaluates the retrieval step of legal RAG systems, which must return precise passages from long legal documents. These benchmarks evaluate one domain each; the present study instead compares strategies by type of question, which allows the results to be mapped onto several domains whose questions differ in type.

D. Evaluation Methodology

Our metrics follow the SQuAD [18] normalization for token-level Exact Match and F_1, and the design of our multi-hop questions follows HotpotQA [19]. For statistical inference we use the percentile bootstrap [20], the paired Wilcoxon signed-rank test [21], and Holm's sequentially rejective correction [22] to control the family-wise error rate across pairwise tests.

III. Research Methodology

A. Research Design

The study is a controlled experiment. The independent variable is the strategy used to supply context to the generator, with seven levels (Section IV-C). A second factor, the generator, has two levels: Gemini 3 Flash for the main experiment and GLM 5.3 Flash for the replication. Everything else is held constant across strategies and generators: the corpus, the 34 questions, the answer prompt, the chunking, the number of retrieved passages, the temperature, the answer-length budget, and the grading. Because the retrieval pipelines share one answer prompt, retrieval is the only part that differs between them.

The dependent measures are token-level Exact Match (EM) and F_1 over all 34 questions (an unanswerable question scores 1 for a refusal and 0 otherwise), refusal accuracy on the unanswerable questions alone, cost in US dollars per full run of the question set, and median and 95th-percentile latency per question. Each strategy is run three times per generator with different seeds, so each strategy receives $34 \times 3 = 102$ graded answers per generator. Seed variance is reported as a measure of stability in its own right. The questions are grouped into five types (Section V-B), and results are reported for each type because the types correspond to the demands of different domains.

B. Stages of the Study

Table I lists the stages in the order in which they were carried out. The questions and gold answers were fixed in S1, before any strategy existed, so that no strategy could influence the test. Stages S2 to S5 built the measuring instrument and checked that it measures correctly: the unit tests confirm that each component behaves as specified, the review in S3 checks the automatic grades against a reading of the answers, and the

TABLE I: Stages of the study. Stages S1 and S6 to S8 contain the experimental work; S2 to S5 build and validate the instrument used to measure it.

Stage	Activity and outcome
S1 Test design	Wrote a four-document corpus and 34 questions with gold answers before any strategy was built; checked every gold answer against the corpus by text search and confirmed that the six unanswerable questions have no answer in it.
S2 Components	Implemented the chunker, dense and BM25 indexes, rank fusion, reranker, grader, and cost ledger, each with unit tests.
S3 Baseline	Implemented Naive RAG, ran it on all 34 questions, and examined each prediction against its automatic grade; most Exact Match misses were correct answers in a different format, so F_1 was adopted as the main metric.
S4 Strategies	Added the six other strategies behind the same interface; tested each end to end with a simulated model, then ran one pilot on the real generator.
S5 Statistics	Added bootstrap intervals, paired Wilcoxon tests, and Holm correction; on the pilots only Recursive Summary differed significantly from the others.
S6 Main experiment	Ran 7 strategies $\times$ 3 seeds at $T = 0.3$ on Gemini 3 Flash (21 runs).
S7 Replication	Ran the same 21-run design on GLM 5.3 Flash; diagnosed the Full-Context failure and ran a follow-up with a larger reasoning budget.
S8 Analysis	Aggregated seeds; computed scores by question type, significance, cost, and variance; derived the recommendations in Section VII.

simulated-model tests in S4 confirm that every strategy runs to completion before any paid call is made. The experimental results come from S6 and S7, and the findings and recommendations from S8.

Runs affected by external failures were discarded and repeated rather than repaired. The first baseline pilot stopped after 23 of 34 questions when the provider's quota was reached and was replaced by a complete run. Two Sliding-Window seeds in S6 were interrupted by an account spending limit; they were set aside and rerun once the limit was lifted. All numbers in this paper come from complete runs.

C. From Results to Recommendations

RQ3 is answered by combining the per-type results with a description of the questions each domain asks. For each domain we identify the question types that dominate it and the constraints that apply (for example, whether the documents fit in the prompt, how much traffic the system receives, and whether a wrong answer is worse than no answer), drawing on the domain benchmarks in Section II where they exist. A strategy is recommended for a domain when it scores well on that domain's dominant question types on *both* generators, so that the recommendation does not rest on one model. Differences that were not statistically significant are used only to break ties between strategies that are otherwise equal, and are described as such. Where a recommendation depends on a property that this study did not measure, it is marked as inferred.

IV. Implementation

This section explains how the seven strategies and the evaluation harness were built, and how the experiments in Section VI were run with them.

A. System Overview

The system is a Python package that runs on a laptop CPU and needs no database server or GPU. A run proceeds as follows. The corpus loader reads the documents, and a sentence-window chunker splits them into chunks of about 150 tokens with a 30-token overlap, which yields 65 chunks for the study corpus. Two indexes are built over the chunks: a dense index in ChromaDB, using `all-MiniLM-L6-v2` [11] embeddings and cosine similarity, and a BM25 [12] index whose tokenizer keeps hyphenated identifiers such as `M7-RX-204-A` intact. The selected strategy then answers each question, calling the generator through a client that records tokens, cost, and latency. Finally, the grader scores each answer, and the analysis tool aggregates the runs into the tables and figures of this paper. Every setting (chunk size, number of retrieved passages, temperature, answer budget, seeds, and spending cap) is read from a configuration file, so a different study changes configuration rather than code.

B. A Common Interface for All Strategies

To make the strategies comparable, every strategy implements the same two-method interface. `setup(chunks, full_text, config)` receives the chunked and unchunked corpus once per run, and `answer(question, qid)` returns the answer together with a trace: an ordered list of steps (retrieval, reranking, generation, summarization) with the tokens, latency, and cost of each. A registry maps strategy names to classes, so the runner, grader, statistics, and studio work with any registered strategy without modification.

Fitting the retrieval pipelines to this interface is direct: `setup` builds or reuses the indexes, and `answer` retrieves passages and makes one generation call. All four use the same answer prompt, which instructs the model to answer in as few words as possible, to use only the supplied passages, and to reply with the exact string `NOT_IN_CONTEXT` when the passages do not contain the answer.

The non-retrieval strategies needed two design decisions to fit the same interface. First, Recursive Summarization does its summarization once per run rather than once per question; its cost is recorded in the trace of the first question, so that the total cost of a run includes this setup work. Second, Sliding-Window Reasoning keeps a running answer between calls; the running answer starts as `NOT_IN_CONTEXT`, so that a question whose answer never appears ends with a refusal. With these decisions every strategy produces the same kind of record, and all seven are graded and costed in the same way.

C. The Seven Strategies

Fig. 1 shows the steps of each strategy. The four retrieval pipelines (R1 to R4) share the chunking, the top-$k = 5$ passages placed in the prompt, and the answer prompt.

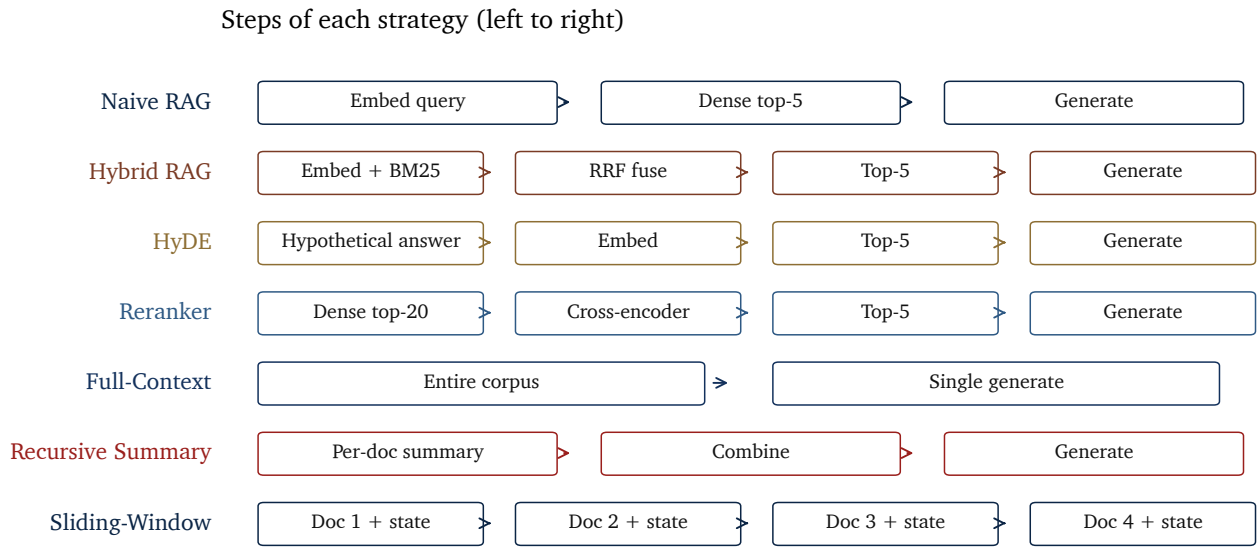


Fig. 1: Steps of the seven strategies. The four retrieval pipelines occupy the upper rows and the three strategies without retrieval the lower rows. All seven implement the same interface (Section IV-B).

a) Naive Vector RAG (R1): The question is embedded with `all-MiniLM-L6-v2`, the five chunks with the highest cosine similarity are placed in the prompt, and one generation call produces the answer. This is the baseline that every other retrieval pipeline must improve on.

b) Hybrid RAG (R2): Dense and BM25 [12] rankings are combined with Reciprocal Rank Fusion (RRF) [7]. For a chunk c at rank $r_i(c)$ in retriever i, the fused score is

$$\mathrm{RRF}(c) = \sum_{i \in \{\text{dense},\text{BM25}\}} \frac{1}{k + r_i(c)}, \quad k = 60, \tag{1}$$

with k set to the value recommended in [7]. Both retrievers are weighted equally, and the five chunks with the highest fused score are used.

c) HyDE (R3): The generator first writes a short, confident passage that would answer the question [3]; the passage is embedded, and the five chunks closest to it are retrieved. The final answer is generated from those chunks with the common answer prompt. HyDE therefore makes two generation calls per question.

d) Cross-Encoder Reranker (R4): The 20 chunks closest to the question are rescored by a cross-encoder (`bge-reranker-base` [9]) that reads the question and chunk together, and the top five after rescoring are used. The reranker runs locally, so the only paid call is the generation call.

e) Full-Context Injection (N1): The entire corpus is placed in the prompt, and one generation call produces the answer. Cost grows linearly with the size of the corpus.

f) Recursive Summarization (N2): Each document is summarized once, with an instruction to preserve every name, date, number, and identifier verbatim, and the concatenated summaries replace the corpus in the answer prompt. The implementation applies this summarization step once; with four documents the concatenated summaries fit in one prompt, so no further level of summarization is needed. The summaries are written without knowledge of the questions.

g) Sliding-Window Reasoning (N3): The corpus is read one document at a time. For each document the model receives the question, the current best answer, and the document, and returns an updated best answer. The four documents give four sequential generation calls per question, and the final state is the answer.

D. Generation Clients and Cost Accounting

All generation goes through a client with a single `generate` call that returns the answer text with its token counts, cost, and latency. Two clients were used. The Gemini client, used for the main experiment, calls Google's API, retries rate-limited calls with exponential backoff, reads token counts from the response metadata, and computes cost from a price table. The OpenRouter client, used for the replication, reaches hosted proprietary and open-weights models through one OpenAI-compatible API, records the cost reported for each call, and sets the reasoning effort of models that reason before answering. It sends only the authorization and content-type headers, so the evaluation traffic carries no application identifier. Adding another provider, including a locally hosted model, requires only the same `generate` call and a price per token.

Every call is written to a cost ledger with its time, strategy, question, input and output tokens, cost, and latency. Before each call the ledger checks the spending cap ($5 in this study) and refuses the call if cumulative spend would exceed 95% of it. The Gemini 3 Flash costs in this paper are read from

this ledger; the GLM 5.3 Flash costs are the per-call amounts reported by OpenRouter and stored with each saved test.

E. Evaluation Harness

The runner executes one strategy on the question set and writes four files for each run: a manifest (configuration hash, dataset hash, code version, and timing), the predictions, the aggregated metrics, and the full trace. A failure on one question is recorded without stopping the run. The grader computes EM and F_1 for answerable questions and refusal accuracy for unanswerable ones (Section V-C). The analysis tool combines the runs of each strategy across seeds, computes bootstrap intervals and pairwise tests, and exports the tables and figures. An automated test suite of more than 100 tests, which replaces the paid model with a simulated one, checks the chunker, indexes, metrics, statistics, each strategy end to end, and the studio.

F. Interactive Studio and Replay

RAG Atlas is a web application built on the same strategy implementations, metrics, and statistics, without modification. Users load documents, choose strategies and a generator, and inspect each retrieval and generation step with its tokens, latency, and cost. Its replay mode loads the study's corpus and questions, runs the chosen strategies on any model available through OpenRouter with the study's settings and seeds, estimates the cost before the first call, and saves the finished test. The GLM 5.3 Flash replication (S7) was run in this mode; an export script converts the saved tests into the tables and figures of Section VI-C.

G. Execution Environment and Reproducibility

All experiments ran on one Apple M2 laptop with 16 GB of memory under macOS, using Python 3.11. Chunking, embedding, indexing, and reranking ran locally on the CPU; only generation used the providers' APIs. The Gemini 3 Flash experiment (S6) ran on 11 and 12 May 2026, and the GLM 5.3 Flash replication (S7) on 11 September 2026. Each run's manifest records the configuration, the hashes of the corpus and question files, and the software versions.

Every table and figure in this paper, including the question-level significance tests and intervals, is regenerated from the raw run files by a single command (`make paper-ieee-regen`). The code, corpus, questions, run files, and saved replication tests are available in the repository given on the first page. The Gemini 3 Flash experiments, including the pilot and discarded runs, cost USD 1.09 in total, and the GLM 5.3 Flash replication and its follow-ups cost USD 0.26.

V. Experimental Setup

A. Corpus

We wrote a four-document fictional corpus of 5,983 words ($\approx$ 8,000 tokens) describing (i) the *Halden Institute for Computational Mycology*, (ii) the *City of Veridian*, (iii) the *MERIDIAN-7 Satellite System*, and (iv) the *Auralite Accord of 1987*, with cross-references between documents that allow genuine multi-hop questions.

A fictional corpus has three properties that are hard to obtain from public benchmarks. First, every gold answer is known to be in the text. Second, distinctive identifiers (for example, `MULBERRY-7Q` and `4.832 GHz`) can be placed at chosen depths to test the recall of single facts. Third, unanswerable questions can be written with certainty that their answer is absent, and the generators cannot answer from memory.

B. Question Types

The 34 questions are divided into five types. **Sanity (4)** questions have a single, easily found answer and check that a strategy works at all. **Needle (8)** questions ask for one specific fact, usually an identifier or number, placed in a single passage. **Multi-hop (8)** questions require combining facts from two or three documents. **Distractor (8)** questions have passages nearby that share their keywords but do not contain the answer. **Hallucination (6)** questions have no answer in the corpus; the correct response is a refusal.

C. Metrics

Token-level EM and F_1 follow the SQuAD normalization [18]: lowercase, remove punctuation and articles, and collapse whitespace. Hyphens in identifiers are replaced with spaces so that, for example, `MULBERRY-7Q` and `MULBERRY 7Q` are treated as equal. For hallucination questions, an answer counts as correct if and only if it is a refusal, detected by matching a curated set of refusal phrases, and both EM and F_1 for these questions equal that correctness; the reported EM and F_1 are means over all 34 questions. Each metric is reported with a percentile-bootstrap 95% confidence interval [20] over $B = 2000$ resamples of the 102 question-seed grades. Because the three grades of a question are correlated, these intervals are narrower than intervals that resample questions: resampling the 34 questions, with seeds averaged, gives intervals about 1.5 to 1.8 times as wide (for Naive RAG on Gemini 3 Flash, $[0.730, 0.920]$ instead of $[0.778, 0.888]$).

D. Statistical Tests

Pairs of strategies are compared with the paired Wilcoxon signed-rank test [21], and the p-values are Holm-corrected [22] across all $\binom{7}{2} = 21$ pairs. The unit of analysis is the question: each strategy contributes 34 F_1 values, each the mean of three seeds, so that correlated seeds are not counted as independent observations.

E. Generators, Seeds, and Budget

Both generators run with the same retrieval and evaluation settings: `all-MiniLM-L6-v2` embeddings, the `bge-reranker-base` cross-encoder, the ChromaDB and BM25 indexes, the same prompts and grading, a 512-token answer budget, temperature 0.3, and three seeds per strategy.

Gemini 3 Flash (`gemini-3-flash-preview`) was called directly through Google's API under a $5 spending cap. Its costs are ledger estimates computed from token counts at an

assumed $0.30 and $2.50 per million input and output tokens, the Gemini 2.5 Flash rates, because the preview model had no fixed price; cost comparisons between the two generators are therefore approximate.

GLM 5.3 Flash (`z-ai/glm-5.3-flash`, with open weights published as `zai-org/GLM-5.3-Flash`) was called through OpenRouter from the studio's replay mode. This model always reasons before answering. The main run requested low reasoning effort with a 640-token limit, which reserves 20% of the limit for reasoning. A Full-Context follow-up used high effort with a 2,560-token limit, which leaves the same 512 tokens for the answer (Section VI-D). Its costs are the amounts OpenRouter reported for each call. We refer to this follow-up as the *headroom run*.

VI. Results

A. Main Experiment on Gemini 3 Flash

Table II reports the main experiment with pooled bootstrap 95% confidence intervals.

The two non-retrieval strategies that read the whole corpus have the highest F_1: Full-Context (0.881) and Sliding-Window (0.875) score above every retrieval pipeline, although their lead is not statistically significant (Section VI-B). HyDE, Naive, and Reranker cluster tightly at F_1 between 0.833 and 0.838; Hybrid follows at 0.797, and Recursive Summary is far below at 0.206. EM lies between 0.627 and 0.696 for the six strategies other than Recursive Summary, a limit set largely by answer formatting rather than retrieval (Section VII-B).

B. Pairwise Significance

On Gemini 3 Flash, Holm-corrected pairwise Wilcoxon tests separate only Recursive Summary, which differs from every other strategy ($p_{\text{adj}} \le 2 \times 10^{-4}$); all other pairs have $p_{\text{adj}} \ge 0.63$. On GLM 5.3 Flash, the Full-Context run with the small reasoning budget differs from the Reranker ($p_{\text{adj}} = 0.005$) and Naive RAG ($p_{\text{adj}} = 0.013$), and no other pair is significant; with the headroom run in its place, no pair is significant. No two retrieval pipelines are separable on either generator. Treating the 102 question-seed grades as independent pairs gives smaller p-values, and on Gemini 3 Flash it also separates Hybrid RAG from Sliding-Window (0.013) and from Full-Context (0.041), but it overstates the sample size. With 34 questions the study cannot order the retrieval pipelines, and the recommendations in Section VII are written with this in mind.

C. Replication on GLM 5.3 Flash

Table III and Fig. 2 compare the two generators. The main result concerns the four retrieval pipelines. Their F_1 differs between the generators by at most 3.8 points (mean 3.1); GLM 5.3 Flash is slightly ahead on Naive, Hybrid, and Reranker and slightly behind on HyDE, and every GLM estimate lies inside the corresponding Gemini 95% CI. Across strategies on a single generator, in contrast, F_1 spans 67.5 points on Gemini 3 Flash and 38.1 points on GLM 5.3 Flash. With the headroom run in place of the first Full-Context run, five of the seven GLM strategies fall inside the Gemini confidence intervals. The fine ordering within the retrieval cluster is not preserved (Spearman $\rho = 0.39$, $p = 0.38$, across all seven strategies, using the headroom run), which is expected because these strategies are not statistically separable on either generator.

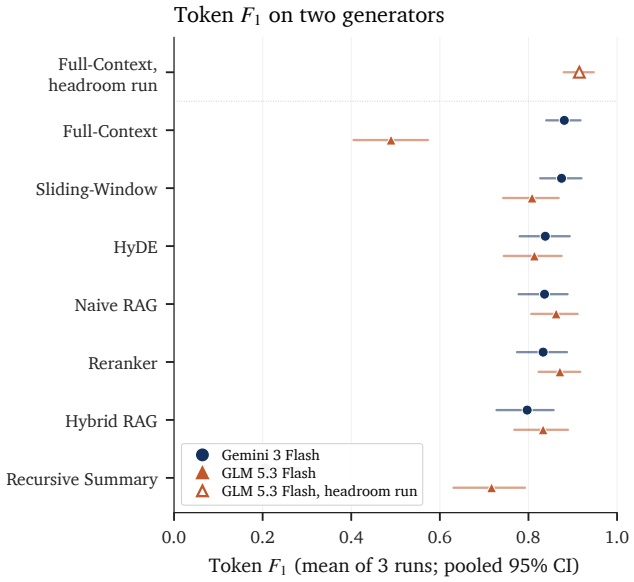


Fig. 2: Token F_1 for each strategy on both generators, with pooled bootstrap 95% CIs over 102 question-seed grades. The four retrieval pipelines overlap almost completely across models. The top row shows the GLM 5.3 Flash headroom run (Section VI-D).

Two strategies depend strongly on the generator. Recursive Summary rises from 0.206 to 0.717: the summaries written by GLM 5.3 Flash keep enough detail for multi-hop and distractor questions (F_1 0.82 and 0.71), although recall of single facts remains poor (0.31). Sliding-Window falls from 0.875 to 0.808 and loses part of its refusal accuracy (Section VI-G). Both are multi-step strategies whose intermediate outputs are themselves model generations, so an error or omission at one step carries into the next.

D. Reasoning Budget for Long Prompts

GLM 5.3 Flash reasons before every answer, and OpenRouter reserves a share of each request's token limit for that reasoning. At low effort the share is 20%, so the 640-token limit reserved about 128 tokens for reasoning. The reservation is a target rather than a hard limit. On the $\approx$ 8,000-token Full-Context prompt the model often reasoned past it: in 11 of the 102 answers the reasoning used the whole 640-token limit and no answer was produced, so the client retried each once with twice the limit. In other answers the model replied `NOT_IN_CONTEXT` for facts that are present (for example, the planted password) or misread numbers ("0.4 ppm" for "0.46 ppm"). The run scored an F_1 of 0.490 overall and 0.24 on needle questions. Repeating the run with high effort and a 2,560-token limit, which leaves the same 512 tokens for the answer, removed every cut-off, used fewer reasoning tokens

TABLE II: Main experiment on Gemini 3 Flash for all seven strategies. Values are means across three seeds at $T = 0.3$ with the standard deviation across seeds. The F_1 CI column gives pooled bootstrap 95% confidence intervals over all $34 \times 3 = 102$ question-seed grades. Cost is the estimated mean USD per full run of the question set.

Strategy	Family	EM ($\mu \pm \sigma$)	F_1 ($\mu \pm \sigma$)	F_1 95% CI	Cost (USD)	p_{50} latency (ms)	p_{95} latency (ms)
Full-Context	non-RAG	0.667 ± 0.017	$\mathbf{0.881 \pm 0.012}$	[0.840, 0.918]	0.0912	18,866	65,349
Sliding-Window	non-RAG	$\mathbf{0.696 \pm 0.045}$	0.875 ± 0.025	[0.826, 0.920]	0.0972	8,317	10,643
HyDE	RAG	0.667 ± 0.017	0.838 ± 0.012	[0.780, 0.893]	0.0186	4,999	7,370
Naive RAG	RAG	0.667 ± 0.017	0.836 ± 0.012	[0.778, 0.888]	0.0122	2,532	7,937
Reranker	RAG	0.667 ± 0.017	0.833 ± 0.009	[0.774, 0.887]	0.0123	2,756	3,786
Hybrid RAG	RAG	0.627 ± 0.017	0.797 ± 0.010	[0.728, 0.857]	0.0122	2,058	7,813
Recursive Summary	non-RAG	0.206 ± 0.000	0.206 ± 0.000	[0.128, 0.284]	0.0068	3,179	12,746

TABLE III: Cross-model replication. Token F_1 on Gemini 3 Flash (Table II) and GLM 5.3 Flash for the same corpus, questions, prompts, and grading, with three seeds at $T = 0.3$ on both. GLM 5.3 Flash used low reasoning effort with a 640-token limit; the last row repeats Full-Context with high effort and a 2,560-token limit, the headroom run (Section VI-D). GLM costs are amounts billed by OpenRouter, whereas Gemini costs are ledger estimates, so costs compare only approximately. Latency is the median per run.

Strategy	Gemini F_1	GLM F_1	GLM F_1 95% CI	ΔF_1	GLM EM	GLM cost (USD)	GLM p_{50} (ms)
Full-Context	0.881 ± 0.012	0.490 ± 0.038	[0.405, 0.573]	-0.391	0.363	0.0094	6,264
Sliding-Window	0.875 ± 0.025	0.808 ± 0.041	[0.743, 0.868]	-0.066	0.647	0.0348	3,026
HyDE	0.838 ± 0.012	0.813 ± 0.017	[0.744, 0.875]	-0.025	0.657	0.0073	2,491
Naive RAG	0.836 ± 0.012	0.862 ± 0.000	[0.806, 0.911]	$+0.026$	0.706	0.0042	697
Reranker	0.833 ± 0.009	0.871 ± 0.007	[0.823, 0.917]	$+0.038$	0.696	0.0051	1,355
Hybrid RAG	0.797 ± 0.010	0.833 ± 0.000	[0.768, 0.889]	$+0.036$	0.676	0.0043	629
Recursive Summary	0.206 ± 0.000	0.717 ± 0.044	[0.631, 0.792]	$+0.511$	0.598	0.0109	765
Full-Context, headroom run	0.881 ± 0.012	$\mathbf{0.915 \pm 0.007}$	[0.880, 0.947]	$+0.034$	0.755	0.0070	1,106

per call (61 against 209, averaged over all calls including retries), and raised F_1 to 0.915 ± 0.007, the highest value on either generator. It also cost less per run ($0.0070 against $0.0094) because no calls had to be retried. The other six strategies used almost no reasoning (between 1 and 15 tokens per call) and were largely unaffected; only HyDE needed two retries. Following this result, the client now reserves at least 1,024 tokens for reasoning by default; a single-run check at low effort under this rule scored $F_1 = 0.931$.

E. Results by Question Type

Fig. 3 reports F_1 by question type on both generators; the recommendations in Section VII rest mainly on this figure. Four observations hold on both generators. First, every retrieval pipeline answers the sanity questions and refuses every unanswerable question. Second, multi-hop is the question type that best separates the retrieval pipelines (from 0.54 to 0.66 on Gemini 3 Flash and from 0.48 to 0.73 on GLM 5.3 Flash); Hybrid RAG is among the weakest pipelines on it (0.54 and 0.56), the Reranker is at or near the top (0.66 and 0.73), and Full-Context is the strongest strategy overall (0.87, and 0.91 in the GLM 5.3 Flash headroom run). Third, the retrieval pipelines' needle scores are nearly the same on both generators (0.83 to 0.88), while their distractor scores are somewhat higher on GLM 5.3 Flash (0.84 to 0.86 against 0.77 to 0.79). Fourth, Recursive Summary is weak on needle questions on both generators (0.00 and 0.31), the type whose answers are the specific identifiers that summaries omit.

F. Cost and Quality

Fig. 4 places the strategies on the cost-quality plane for both generators. On Gemini 3 Flash, Naive, Hybrid, and Reranker RAG cost about $0.012 per run and HyDE, with its second call, $0.019; Full-Context and Sliding-Window cost 7.5 and 8.0 times as much as Naive RAG for an F_1 gain of about 0.04. On GLM 5.3 Flash the retrieval pipelines cost between $0.004 and $0.007 per run, and the headroom run reaches the highest F_1 at a comparable cost ($0.0070), because the long prompt is billed at a low input price. The cost premium of reading the whole corpus therefore depends on the generator's input price. Summed over the seven strategies, a GLM 5.3 Flash run cost roughly one third of the estimated Gemini 3 Flash run; because one figure is billed and the other estimated, the ratio is indicative only.

G. Seed Variance and Refusal

Fig. 5 shows that single-call retrieval pipelines are close to deterministic at $T = 0.3$ on both generators (Naive and Hybrid RAG have zero seed variance on GLM 5.3 Flash), while multi-step strategies accumulate variance: Sliding-Window reaches $\sigma_{F_1} = 0.025$ on Gemini 3 Flash and 0.041 on GLM 5.3 Flash, and Recursive Summary 0.044 on GLM 5.3 Flash. Recursive Summary has zero variance on Gemini 3 Flash because it failed in the same way on every seed. Fig. 6 separates correctness into factoid recall and refusal. All four retrieval pipelines and Recursive Summary refuse every unanswerable question on both generators; Sliding-Window refuses only two thirds of them on GLM 5.3 Flash, and the Full-Context run with the small reasoning budget slightly fewer than all.

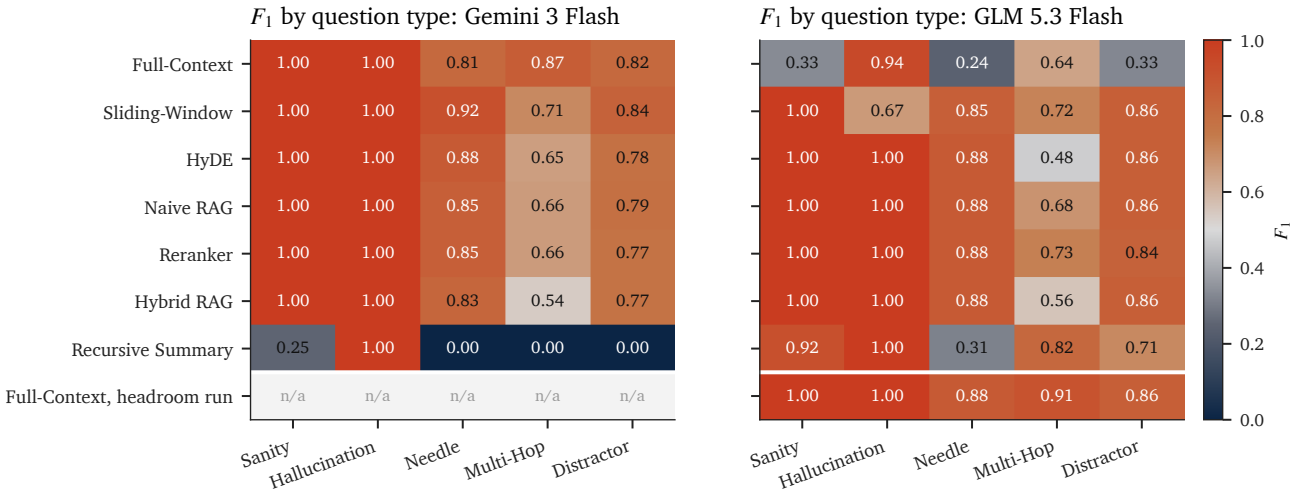


Fig. 3: Token F_1 by question type on Gemini 3 Flash (left) and GLM 5.3 Flash (right), with the same row order. The bottom row adds the GLM 5.3 Flash headroom run. The recommendations in Section VII are derived from these values.

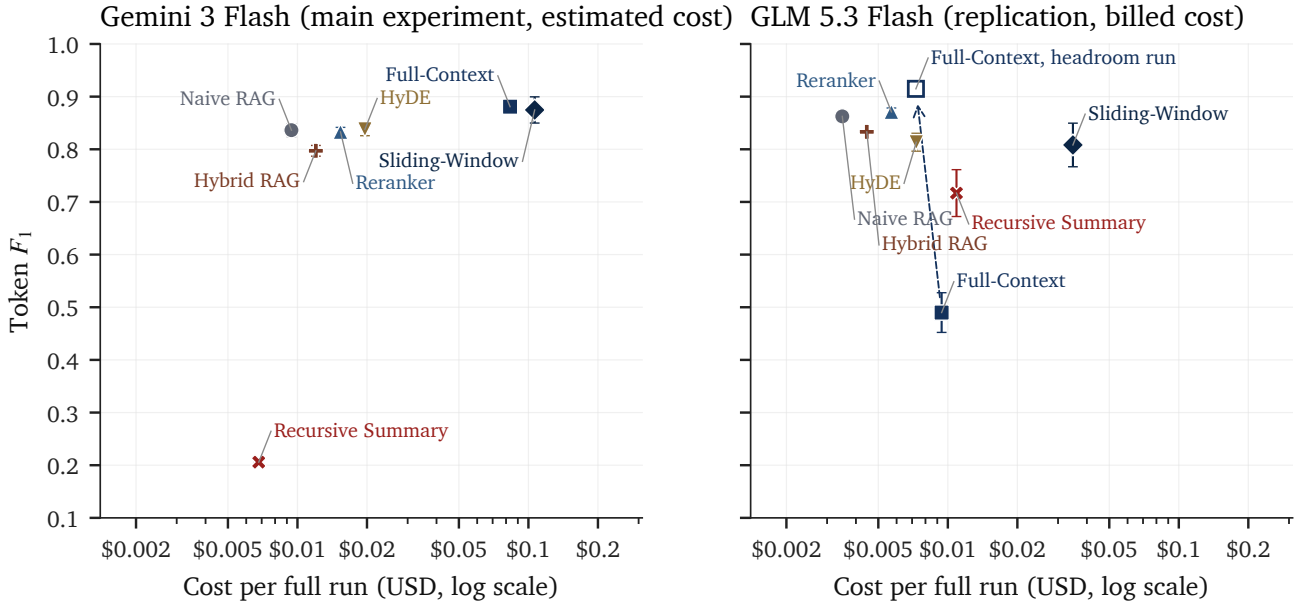


Fig. 4: Cost per full run (log scale) against token F_1 on each generator; error bars are seed standard deviations. Gemini 3 Flash costs are ledger estimates and GLM 5.3 Flash costs are billed amounts. Points that would coincide are moved slightly apart horizontally for legibility; exact costs are in Tables II and III. The dashed arrow marks the GLM 5.3 Flash headroom run.

VII. Research Findings and Recommendations

This section states what the study found, answers the three research questions, and turns the results into recommendations.

A. Answers to the Research Questions

RQ1: Comparison of strategies: When the corpus fits in the prompt, the strategies that read the whole corpus have the highest F_1: Full-Context leads on both generators, and Sliding-Window nearly matches it on Gemini 3 Flash at a similar cost. The four retrieval pipelines score 4 to 8 points of F_1 lower on Gemini 3 Flash and 4 to 10 points lower than the headroom run on GLM 5.3 Flash, at one fifth to one seventh of the cost on Gemini 3 Flash. These gaps are not statistically significant with 34 questions, and the retrieval pipelines cannot be ordered among themselves. The advantage of reading the whole corpus comes mainly from multi-hop questions; on single-fact and distractor questions, Full-Context and the retrieval pipelines score within 7 points of each other on both generators. Apart from the budget-limited Full-Context run, question-agnostic summarization is the only strategy that fails outright on single-fact questions, and it is the only strategy that differs significantly from all others on Gemini 3 Flash.

RQ2: Dependence on the generator: The retrieval pipelines transfer between the two generators: F_1 changes by at most 3.8 points, inside the confidence intervals, while the

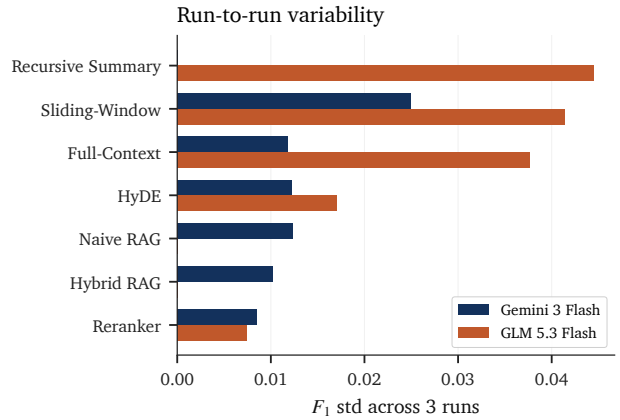


Fig. 5: F_1 standard deviation across three seeds on both generators. Sliding-Window has the largest variance on Gemini 3 Flash, and Recursive Summary (0.044) and Sliding-Window (0.041) on GLM 5.3 Flash; single-call retrieval pipelines are nearly deterministic.

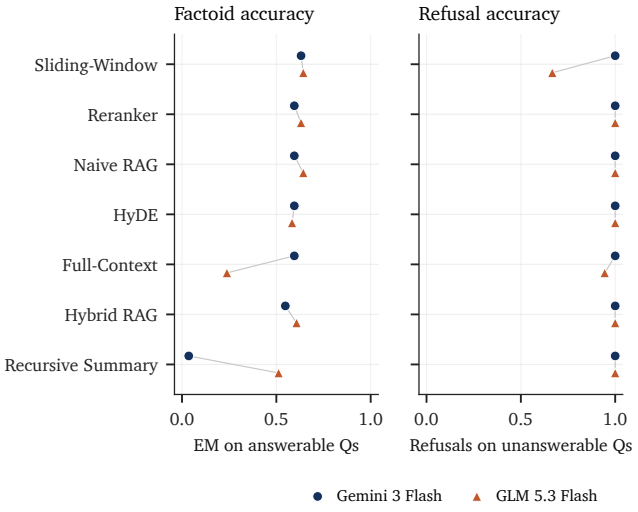


Fig. 6: Factoid accuracy (EM on answerable questions) and refusal accuracy (on unanswerable questions) for each strategy, with one marker per generator on each row. The retrieval pipelines refuse every unanswerable question on both generators.

choice of strategy changes F_1 by up to 67.5 points on one generator. The strategies whose later steps depend on earlier model outputs (Recursive Summary and Sliding-Window) and the strategy with the longest prompt (Full-Context) depend on the model. For a model that always reasons, the Full-Context result depends on the reasoning-token budget as much as on the model itself.

RQ3: Strategy by use case: Table IV and Section VII-D give the recommendations. In short, Full-Context is preferred when the relevant documents fit in the prompt and questions often combine several passages; a single-call retrieval pipeline, with the Cross-Encoder Reranker as the default, is preferred otherwise; and question-agnostic summarization should not be used for fact lookup.

B. Principal Findings

Finding 1: The strategy matters more than the generator: For the retrieval pipelines, switching between a proprietary and an open-weights flash-tier generator moved F_1 by at most 3.8 points, while switching strategy on one generator moved it by up to 67.5 points. A likely reason is that these pipelines leave the model a narrow task: the answer is in one of five short passages, and the model only has to extract it. For a given corpus and question mix, the choice of pipeline is therefore the larger decision; the generator can then be chosen on cost, latency, openness, or deployment constraints and checked with a replay. This finding is limited to two generators of similar size and one corpus.

Finding 2: Reading the whole corpus helps most on multi-hop questions: At the 8,000-token scale tested here, placing the corpus in the prompt gave the highest F_1 on both generators. The overall gap to the retrieval pipelines is small (≈ 0.04) and not statistically significant, but it has the same sign on both generators and is concentrated in multi-hop questions (0.87 against 0.54 to 0.66 on Gemini 3 Flash, and 0.91 against 0.48 to 0.73 on GLM 5.3 Flash), where retrieval must find every relevant passage among its top five while Full-Context sees all of them. For a model that always reasons, the advantage appears only when the reasoning budget is set separately from the answer budget (Section VI-D); with a small token limit, the same strategy had the lowest F_1 of all seven.

Finding 3: Question-agnostic summarization loses specific facts: Recursive Summary's needle F_1 was 0.00 and 0.31 on the two generators: summaries written without knowledge of the questions omit the identifiers and numbers that single-fact questions ask about, even when instructed to keep them. Its overall F_1, however, ranged from 0.206 to 0.717, so a capable summarizer recovers much of the multi-hop and distractor performance. Summarization should not replace retrieval for fact lookup, but it depends on the model enough that applications built on synthesis should measure it before ruling it out.

Finding 4: Refusal can be set in the prompt for single-call pipelines: The strict refusal instruction gave a refusal accuracy of 1.000 for all four retrieval pipelines on both generators. Multi-step strategies are less reliable: Sliding-Window's refusal accuracy fell to 0.667 on GLM 5.3 Flash, probably because each intermediate step is another chance to replace the refusal with a guess. Refusal should be measured again whenever intermediate model outputs feed later steps.

Finding 5: Exact Match mostly measures answer formatting: On Gemini 3 Flash, EM lies between 0.627 and 0.696 for the six strategies other than Recursive Summary, while their F_1 ranges from 0.797 to 0.881. The review in S3 and the GLM 5.3 Flash answers show that many EM misses are correct answers in another format, such as "Twenty-five years" for the gold answer "25 years" and "Seven years" for "7 years". Because answer style differs between generators, comparisons of short-answer question answering across models should report F_1 alongside EM.

Finding 6: Multi-step strategies need several seeds: The multi-step strategies had the largest seed variance: Sliding-Window reached $\sigma_{F_1} = 0.025$ on Gemini 3 Flash, about twice the 0.009 to 0.012 of the retrieval pipelines, and on GLM 5.3 Flash Recursive Summary and Sliding-Window reached 0.044 and 0.041, against at most 0.017 for the retrieval pipelines. The budget-limited Full-Context run (0.038) is the only single-call exception, and Recursive Summary had zero variance on Gemini 3 Flash because it failed in the same way on every seed. A single run of a multi-step strategy can misstate its performance by a few points: one Sliding-Window run on Gemini 3 Flash scored 0.896 against a three-seed mean of 0.875.

C. Research Contributions

The study adds four pieces of evidence to the RAG literature. First, it measures seven strategies under matched conditions and separates the results by question type, which shows where each strategy succeeds and fails rather than only its average score. Second, it shows on two generators that the retrieval pipelines transfer between models while multi-step and long-prompt strategies do not, which tells a practitioner which results must be measured again when the model changes. Third, it identifies the reasoning-token budget as a setting that moved Full-Context from the lowest F_1 to the highest on one generator, an interaction between long prompts and reasoning models that must be controlled when such models are evaluated. Fourth, it converts these results into recommendations for specific domains, stating the evidence for each. The software written for the study is a means to these results; it is released so that other researchers can test the findings on their own corpora and generators.

D. Recommendations by Use Case

Table IV summarizes the recommendations; the reasoning behind each follows.

a) Medical question answering: Medical questions ask for specific drug names, doses, codes, and contraindications, which are needle questions, and a fabricated dose can harm a patient, so refusal is essential. MedRAG [15] reports that the choice of retriever changes accuracy substantially on medical benchmarks. In this study every retrieval pipeline recalled single facts about equally well on both generators and refused every unanswerable question. The Reranker is suggested as the default because it was at or near the top on multi-hop questions on both generators at almost the same cost as Naive RAG; the difference is not significant, and Naive RAG is an equally reasonable choice when latency matters. Full-Context recalled single facts as well as retrieval on GLM 5.3 Flash (0.88) and slightly worse on Gemini 3 Flash (0.81), so it is an option when the patient record fits in the prompt and questions combine several parts of it, with a sufficient reasoning budget for models that reason. Sliding-Window had the highest needle score on Gemini 3 Flash (0.92) but lost a third of its refusals on GLM 5.3 Flash, and Recursive Summary loses exactly the specifics that medical answers require.

b) Legal research and citation: Legal work requires verbatim recall of statutes and precedents, multi-hop reasoning across clauses, and resistance to similar but irrelevant text, such as a clause from a different statute. LegalBench [16] and LegalBench-RAG [17] emphasize precise retrieval of passages from long legal documents. Full-Context led on multi-hop questions on both generators. Hybrid RAG with equal weights was among the weakest pipelines on them on both generators, consistent with BM25 favoring passages that share words with the question but not its meaning; the difference is not significant, but it makes Hybrid the last choice among the retrieval pipelines for this use case. Legal applications should also evaluate quotation accuracy, which our question types do not measure.

c) Education and tutoring: Tutoring questions often combine material from several chapters, answers are explanations rather than exact spans, and traffic is high, so cost per answer matters. Naive RAG was among the cheapest pipelines on both generators, and the Reranker cost almost the same while scoring higher on multi-hop questions with GLM 5.3 Flash; both refused every unanswerable question. Recursive Summary scored 0.82 on multi-hop questions with GLM 5.3 Flash but 0.00 with Gemini 3 Flash, so it is an option for synthesis questions only after it has been measured on the chosen model. Pure F_1 does not measure the quality of an explanation, so a rubric-based evaluation should be added.

d) Finance and regulatory filings: Financial questions combine the needle and refusal profile of medicine (specific figures, dates, and percentages) with the multi-hop profile of law (reasoning across filings). The Reranker is recommended, with an answer-format instruction for numbers, and Full-Context when a single filing fits in the prompt. Recursive Summary should be avoided because specific figures are the details that summaries omit.

e) Internal knowledge bases: An internal wiki, FAQ collection, or product handbook often fits in a modern context window. Full-Context is then the strongest option, provided models that reason are given a separate reasoning budget; the Reranker is the fallback when cost per query is the binding constraint. The cost premium of Full-Context depends on the generator: 5 to 7.5 times that of the retrieval pipelines on Gemini 3 Flash, but about the same as HyDE and 1.4 to 1.7 times the other pipelines on GLM 5.3 Flash.

f) Software documentation and code: Code differs from prose: useful units are functions and classes rather than sentences, and dense embeddings trained on prose represent exact symbol names poorly. Hybrid RAG with a higher BM25 weight than the equal weighting used here is the reasoned choice, but this recommendation is inferred and was not tested.

E. Selection Procedure

The findings can be combined into a procedure for choosing a strategy:

1) If the documents needed to answer a question fit in the model's context window, the cost per query is acceptable, and questions often combine information from several

TABLE IV: Recommended strategies by use case. *Dominant questions* are the question types that characterize the use case. *Evidence* cites F_1 from Fig. 3 as Gemini 3 Flash / GLM 5.3 Flash, with Full-Context on GLM 5.3 Flash taken from the headroom run. *Basis*: M = measured in this study; I = inferred, not measured. Differences among the retrieval pipelines are not statistically significant and are used only to order otherwise acceptable options.

Use case	Dominant questions	Recommended	Avoid	Evidence	Basis
Medical question answering	Needle; refusal	Reranker or Naive RAG; Full-Context when the record fits	Recursive Summary; Sliding-Window unless refusal is re-measured	Needle: retrieval 0.83 to 0.88 / 0.88, Full-Context 0.81 / 0.88; retrieval refusal 1.00 on both; Recursive Summary needle 0.00 / 0.31; Sliding-Window refusal 1.00 / 0.67	M
Legal research and citation	Multi-hop; distractor	Full-Context within one statute or contract; Reranker for larger collections	Recursive Summary	Multi-hop: Full-Context 0.87 / 0.91, Reranker 0.66 / 0.73, Hybrid 0.54 / 0.56	M
Education and tutoring	Multi-hop synthesis; high volume	Naive RAG or Reranker	Recursive Summary unless measured on the chosen model	Naive and Reranker cost \$0.012 / \$0.004 to \$0.005 per run; Recursive Summary multi-hop 0.00 / 0.82	M
Finance and regulatory filings	Needle (figures); multi-hop; refusal	Reranker; Full-Context for a single filing	Recursive Summary	As for medical and legal; figures are needle questions	M
Internal knowledge base that fits in the prompt	Mixed	Full-Context with a sufficient reasoning budget; Reranker when cost is binding	Full-Context with a small token limit on a reasoning model	Full-Context 0.881 / 0.915, but 0.490 with a small limit; cost 5 to 7.5× that of retrieval on Gemini, 1.0 to 1.7× on GLM	M
Software documentation and code	Exact identifiers	Hybrid with a higher BM25 weight	None measured	Not tested: code corpora and weighted fusion were outside this study	I

passages, use Full-Context. For a model that reasons, set the reasoning budget separately from the answer budget.

2) Otherwise, use a single-call retrieval pipeline. The differences among them were not significant; the Cross-Encoder Reranker is a reasonable default because it was at or near the top on multi-hop questions on both generators, and Naive RAG is the simplest choice when cost and latency dominate.
3) Do not use question-agnostic summarization when users ask for specific names, numbers, or identifiers.
4) When refusal matters, use a single-call pipeline with a strict refusal instruction, and measure refusal again for any multi-step strategy.
5) Before deployment, repeat the comparison on a sample of real questions, preferably 200 or more, grouped by the question types above, with at least three seeds for multi-step strategies.

VIII. Limitations and Future Work

a) Corpus scale: The 8,000-token corpus is small compared with many production collections. Retrieval is expected to regain its advantage once the corpus exceeds the context window or becomes long enough for evidence to be lost in the middle of the prompt [14]. Measuring where this crossover occurs is the most important next study.

b) Sample size: With 34 questions and Holm correction across 21 pairs, only large differences are statistically detectable, and the retrieval pipelines could not be ordered. A larger question set, on the order of 200 or more, is needed to resolve smaller differences.

c) Generators: The study covers two flash-tier generators, one proprietary and one open-weights. The transfer of the retrieval pipelines may not hold for much smaller models, models with weaker attention over long inputs, or models with very different answer styles. Replications on frontier-scale, small, and locally hosted open-weight models [23] are needed.

d) Reasoning settings: Models that reason add reasoning effort and token limits as experimental variables. We report the settings of every run; a systematic study of reasoning effort against strategy and corpus length is future work.

e) Strategies not studied: Self-RAG [6], GraphRAG [4], RAPTOR [5], parent-child retrieval, multi-query retrieval, contextual compression, and agent-based memory architectures were not evaluated. Each can be added through the interface in Section IV-B and compared under the same conditions.

f) Question types and metrics: The five question types do not cover everything production users care about. Quotation accuracy (legal), citation accuracy (academic), arithmetic (finance), and quality of explanation (education) are natural additions, and token F_1 does not measure the quality of long answers.

g) Temperature: The study runs at $T = 0.3$, which is close to deterministic for short answers. Higher temperatures will increase seed variance and may affect refusal.

h) Authorship of the questions: The questions were written by the same researchers who selected the strategies. Replication with questions written by independent annotators would remove this possible bias; the question files are plain JSONL and easy to audit.

IX. Conclusion

This paper compared seven strategies for supplying documents to a language model, four retrieval pipelines and three strategies without retrieval, under matched conditions on a controlled corpus, with results broken down by type of question and replicated on a proprietary and an open-weights generator.

Three results stand out. First, the choice of strategy mattered far more than the choice between the two generators: the retrieval pipelines gave nearly identical results on both models, while strategies differed by up to 67.5 points of F_1 on one model. Second, when the corpus fits in the prompt, reading the whole corpus gave the highest F_1 on both generators, mainly through better answers to multi-hop questions, provided a reasoning model is given a separate reasoning budget; with 34 questions this lead over retrieval is not statistically significant. Third, question-agnostic summarization lost the specific facts that many domains depend on, and multi-step strategies were less stable and less reliable at refusing than single-call retrieval. From these results we derived recommendations for six application areas and a general procedure for choosing a strategy, stating the evidence behind each. The study is limited to one small corpus, 34 questions, and two generators; the evaluation software, corpus, questions, and run files are released so that these findings can be tested at larger scale, in other domains, and with other models.

Acknowledgments

We thank the maintainers of `sentence-transformers`, `ChromaDB`, `rank_bm25`, and the BAAI FlagEmbedding project, whose freely available components made this study possible at low cost.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[3] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise zero-shot dense retrieval without relevance labels," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 1762–1777.

[4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A Graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[5] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive abstractive processing for tree-organized retrieval," in *International Conference on Learning Representations (ICLR)*, 2024.

[6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *International Conference on Learning Representations (ICLR)*, 2024.

[7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.

[8] R. Nogueira and K. Cho, "Passage re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.

[9] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packed resources for general Chinese embeddings," *arXiv preprint arXiv:2309.07597*, 2023.

[10] Gemini Team, Google, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *arXiv preprint arXiv:2403.05530*, 2024.

[11] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.

[12] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[13] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 39–48.

[14] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[15] G. Xiong, Q. Jin, Z. Lu, and A. Zhang, "Benchmarking retrieval-augmented generation for medicine," in *Findings of the Association for Computational Linguistics (ACL)*, 2024, pp. 6233–6251.

[16] N. Guha, J. Nyarko, D. E. Ho, C. Ré *et al.*, "LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[17] N. Pipitone and G. Houir Alami, "LegalBench-RAG: A benchmark for retrieval-augmented generation in the legal domain," *arXiv preprint arXiv:2408.10343*, 2024.

[18] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2016, pp. 2383–2392.

[19] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 2369–2380.

[20] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[21] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[22] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[23] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.